

Platform Evaluation User Guide for Intel® Multiple GPU Solution

User Guide

September 2025

Document Number: 863931-1.0



Notice: This document contains information on products in the design phase of development. The information here is subject to change without notice. Do not finalize a design with this information.

Intel technologies may require enabled hardware, software or service activation.

You may not use or facilitate the use of this document in connection with any infringement or other legal analysis concerning Intel products described herein. You agree to grant Intel a non-exclusive, royalty-free license to any patent claim thereafter drafted which includes subject matter disclosed herein.

No license (express or implied, by estoppel or otherwise) to any intellectual property rights is granted by this document.

The products described may contain design defects or errors known as errata which may cause the product to deviate from published specifications. Current characterized errata are available on request.

Performance results are based on testing as of dates shown in configurations and may not reflect all publicly available updates. See backup for configuration details. No product or component can be absolutely secure.

Performance varies by use, configuration, and other factors. Learn more on the Performance Index site.

Your costs and results may vary.

“Conflict-free" refers to products, suppliers, supply chains, smelters, and refiners that, based on our due diligence, do not contain or source tantalum, tin, tungsten or gold (referred to as “conflict minerals" by the U.S. Securities and Exchange Commission) that directly or indirectly finance or benefit armed groups in the Democratic Republic of the Congo or adjoining countries.

All product plans and roadmaps are subject to change without notice.

Code names are used by Intel to identify products, technologies, or services that are in development and not publicly available. These are not "commercial" names and not intended to function as trademarks.

Intel disclaims all express and implied warranties, including without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement, as well as any warranty arising from course of performance, course of dealing, or usage in trade.

(Optional-Overclocking Disclaimer) Altering clock frequency or voltage may void any product warranties and reduce stability, security, performance, and life of the processor and other components. Check with system and component manufacturers for details.

(Optional ENERGY STAR) ENERGY STAR is a system-level energy specification, defined by the US Environmental Protection Agency, that relies on all system components, such as processor, chipset, power supply, and so forth. For more information, visit Energy Star.

(Optional Intel® Thermal Velocity Boost (Intel® TVB)) Includes the effect of Intel Thermal Velocity Boost, a feature that opportunistically and automatically increases clock frequency above single-core and multi-core Intel Turbo Boost Technology frequencies based on how much the processor is operating below its maximum temperature and whether turbo power budget is available. The frequency gain and duration are dependent on the workload, capabilities of the processor and the processor cooling solution.

(Optional Intel® Tubo Boost Technology) Intel® Turbo Boost Technology requires a PC with a processor with Intel Turbo Boost Technology capability. Intel Turbo Boost Technology performance varies depending on hardware, software, and overall system configuration. Check with your PC manufacturer on whether your system delivers Intel Turbo Boost Technology. For more information, see http://www.intel.com/technology/turboboost

(Optional-Estimated Results Disclaimer) Results have been estimated or simulated.

(Optional-Third-party Data Disclaimer) Intel does not control or audit third-party data. You should consult other sources to evaluate accuracy.



(Optional - Principled Technologies Benchmark Discloser) Intel contributes to the development of benchmarks by participating in, sponsoring, and/or contributing technical support to various benchmarking groups, including the BenchmarkXPRT Development Community administered by Principled Technologies.

Copies of documents which have an order number and are referenced in this document may be obtained by calling 1-800548-4725 or visiting the Intel Resource and Document Center.

© Intel Corporation. Intel, the Intel logo, and other Intel marks are trademarks of Intel Corporation or its subsidiaries. Other names and brands may be claimed as the property of others.

Contents

1.0 Introduction 7
1.1 Prerequisite 7
1.2 Terminology 7
2.0 BIOS Configuration 9
2.1 Disable PCIe ACS 9
2.2 Disable IOMMU 10
3.0 Software Installation 12
4.0 P2P Tests 13
4.1 Query Capability 13
4.2 Basic Test 14
4.3 Parallel Test 16
4.4 Stability Test 18
5.0 Memory Bandwidth Tests 19
5.1 H2D/D2H Tests 19
5.2 D2D Tests 20
6.0 oneCCL Tests 22
6.1 Options 22
6.2 Command Examples 23
7.0 GEMM Tests 26
7.1 matrix_mul_mkl usage 26
7.2 Run matrix_mul_mkl 26

Figures

Figure 1. Disable PCIe ACS in BIOS 9
Figure 2. Check ACS Status in Ubuntu 10
Figure 3. Disable IOMMU in OS 11
Figure 4. Check dmesg for Disabled IOMMU 11
Figure 5. ze_peer Query Output 13
Figure 6. Run ze_peer Default Test 14
Figure 7. parallel_pair_targets Test with 3 Pairs 17
Figure 8. Run Stability Test with AllReduce-like Operation 18
Figure 9. Known Issue with “-c” Option 18
Figure 10. ze_peak Help Menu 19
Figure 11. ze_peak transfer_bw result 19
Figure 12. PCIe Bandwidth Calculation 20

Figure 13. ze_peak global_bw result 21
Figure 14. allreduce results with 4 B60 Pro 23
Figure 15. allgather results with 4 B60 Pro 24
Figure 16. alltoall results with 4 B60 Pro 25
Figure 17. matrix_mul_mkl usage 26
Figure 18. matrix_mul_mkl on B60 with int8 27

Tables

Table 1. Terminology 7
Table 2. ze_peer Command Option 14
Table 3. Parallel Test Options 16
Table 4. ze_peak GPU Copy Details 20
Table 5. oneCCL benchmark Command Option 22

Revision History

Date	Revision	Description
September 2025	1.0	Initial release for B60 Pro.

§

1.0 Introduction

GPU computing and cross-GPU communication capabilities are critical for the Intel Multi-GPU inference system. This collateral covers the user guide of a couple of evaluation toolkits on Intel Battlemage GPU to benchmark above 2 capabilities of a system, including GPU memory bandwidth (H2D - Host to GPU Device, D2H – GPU Device to Host, D2D – GPU Device to Device), P2P/Collective Communication over PCIe, and General Matrix Multiplication.

1.1 Prerequisite

Refer to the Intel Multi-GPU Get Started Guide for the B60 Pro GPU to set up the environment. All required tools would be ready both in bare metal and docker environments after the installation.

1.2 Terminology

Table 1. Terminology

Term	Description
ACS	Access Control Service
AMI*	American Megatrends Inc. (BIOS Vendor)
API	Application Programming Interface
BIOS	Basic Input/Output System
BW	Bandwidth
CPU	Central Processing Unit
GPU	Graphics Processing Unit
IOMMU	Input/Output Memory Management Unit
LLM	Large Language Model
oneCCL	oneAPI Collective Communications Library
oneMKL	oneAPI Math Kernel Library
OS	Operating System
P2P	Peer-to-Peer

Term	Description
PCIe	Peripheral Component Interconnect Express
UPI	Ultra Path Interconnect

§

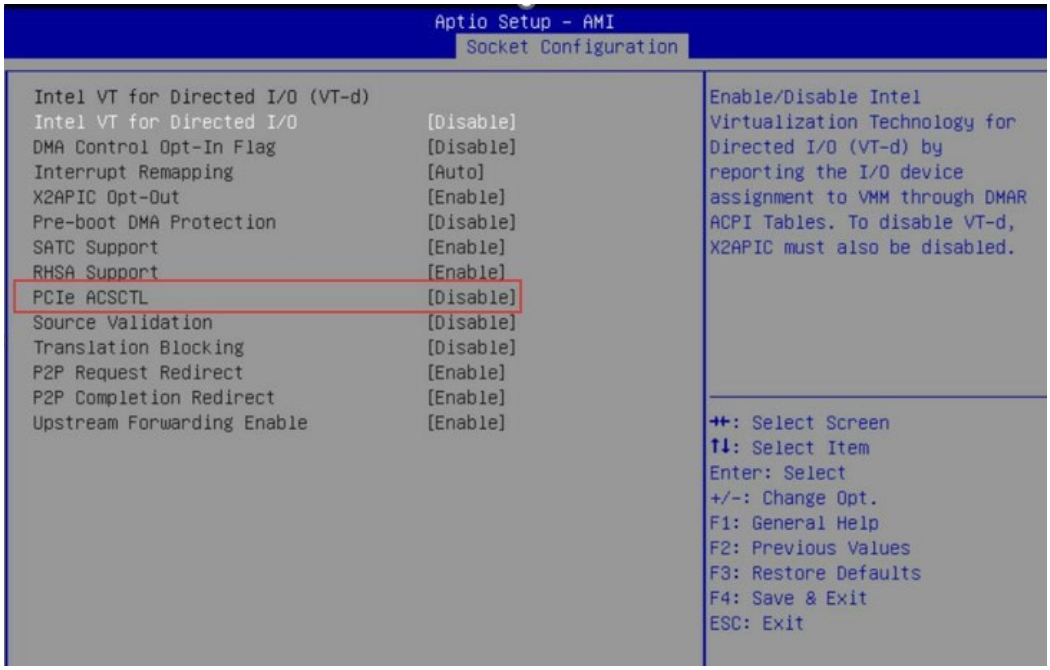
2.0 BIOS Configuration

Some system features would have a significant performance impact on P2P communication. Suggest disabling the PCIe ACS (Access Control Service) and IOMMU features. ACS would route all the data transfer back to the root complex. IOMMU would introduce unnecessary address translation. Both features would block the P2P transfer.

2.1 Disable PCIe ACS

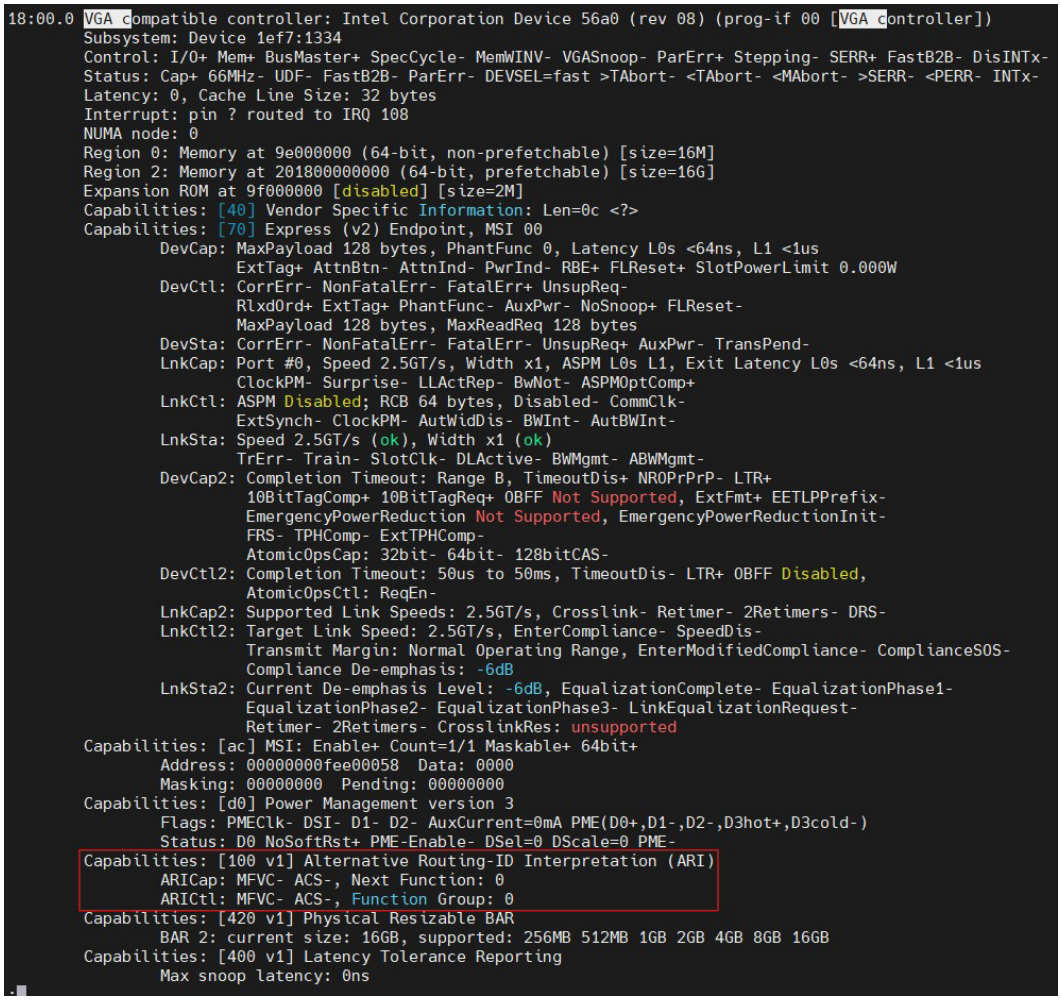
PCIe ACS can be disabled in the BIOS. Below is an example from the AMI BIOS.

Figure 1. Disable PCIe ACS in BIOS



As a result, once you see “ACS-” in lspci -vvv under Ubuntu from your GPU device as below (ARICtl part), this feature is disabled.

Figure 2. Check ACS Status in Ubuntu

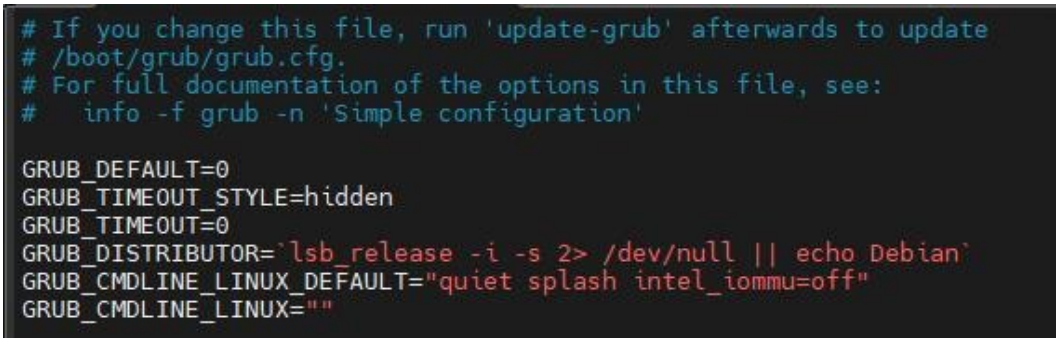


2.2 Disable IOMMU

You can disable the IOMMU features in the OS with the intel_iommu=off option in the grub. Remember to update-grub and reboot after you edit /etc/default/grub to make sure this option is effective.

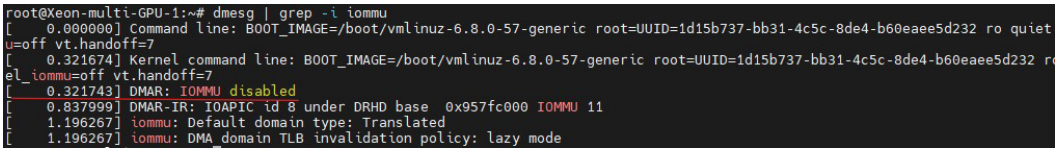
If you set up the environment following the Intel Multi-GPU Get Started Guide, the IOMMU feature would be disabled during the installation.

Figure 3. Disable IOMMU in OS



You can confirm whether IOMMU is disabled with dmesg information (IOMMU disabled).

Figure 4. Check dmesg for Disabled IOMMU



§

3.0 Software Installation

In this guide, P2P & GPU Memory Bandwidth evaluations are based on the Intel level-zero-tests project. Specifically, we use the tool “ze_peer” for P2P and “ze_peak” for memory bandwidth benchmark from this project: https://github.com/oneapi-src/level-zero-tests.

oneCCL toolkit supports AllReduce/AllGather/All2All operation based on Intel oneCCL library.

General Matrix Multiply is based on Intel oneMKL test sample from oneAPI.

Once you complete the installation by following the above Get Started Guide, you will get these tools in the platform evaluation docker image. GEMM/OneCCL tools are installed in /usr/bin. Level-zero-tests is under /opt/intel/multi-arc/multi-arc-bmg-offline-installer-x.x.x.x/tools/level-zero-tests directory.

§

4.0 P2P Tests

ze_peer can utilize different engines inside the GPU to complete the P2P communication with different package sizes, data direction, and operation patterns.

4.1 Query Capability

"-q" option can Query the engines available in the devices. Numbers under the `-u` column can be used to select a set of engines. In the sample below, two B60 Pro devices were discovered, each one having 2 engines: 1 with compute and copy capabilities, and 1 with only copy capabilities.

Figure 5. ze_peer Query Output

```
root@edgeaihost10:~# ./ze_peer -q
=====================================================
 Device 0
        Intel(R) Graphics [0xe211]
        2400 MHz
-----------------------------------------------------
| Group  | Number of Queues | Compute | Copy |  -u |
=====================================================
|   0    |         1        |    X    |   X  |     |
|        |                  |         |      |   0 |
-----------------------------------------------------
|   1    |         1        |         |   X  |     |
|        |                  |         |      |   1 |
-----------------------------------------------------
=====================================================
=====================================================
 Device 1
        Intel(R) Graphics [0xe211]
        2400 MHz
-----------------------------------------------------
| Group  | Number of Queues | Compute | Copy |  -u |
=====================================================
|   0    |         1        |    X    |   X  |     |
|        |                  |         |      |   0 |
-----------------------------------------------------
|   1    |         1        |         |   X  |     |
|        |                  |         |      |   1 |
-----------------------------------------------------
=====================================================
```

Engines under the same group have the same capabilities. The same capability from engines under different groups may be different. For example, the copy capability of engine 0 and 1 would be different.

4.2 Basic Test

By default, unidirectional transfer bandwidth and latency tests are executed for sizes between 8 B and 256 MB for write and read operations between all devices detected in the system using the first compute engine in the device.

Figure 6. Run ze_peer Default Test

```
root@Xeon-multi-GPU-1:/home/zhonghua/level-zero-tests/build/perf_tests/ze_peer# ./ze_peer
================================================================================
Single Process Unidirectional tests
================================================================================
--------------------------------------------------------------------------------
Bandwidth Write : Device( 0 )->Device( 1 )
BW [GBPS]:     8  B:      0.00
BW [GBPS]:    16  B:      0.00
BW [GBPS]:    32  B:      0.00
BW [GBPS]:    64  B:      0.00
BW [GBPS]:   128  B:      0.01
BW [GBPS]:   256  B:      0.01
BW [GBPS]:   512  B:      0.02
BW [GBPS]:     1 KB:      0.05
BW [GBPS]:     2 KB:      0.09
BW [GBPS]:     4 KB:      0.18
BW [GBPS]:     8 KB:      0.34
BW [GBPS]:    16 KB:      0.70
BW [GBPS]:    32 KB:      1.59
BW [GBPS]:    64 KB:      2.79
BW [GBPS]:   128 KB:      4.42
BW [GBPS]:   256 KB:      6.53
BW [GBPS]:   512 KB:      8.41
BW [GBPS]:     1 MB:     10.92
BW [GBPS]:     2 MB:     11.32
BW [GBPS]:     4 MB:     11.93
BW [GBPS]:     8 MB:     17.11
BW [GBPS]:    16 MB:     21.14
BW [GBPS]:    32 MB:     21.85
BW [GBPS]:    64 MB:     21.68
BW [GBPS]:   128 MB:     22.05
BW [GBPS]:   256 MB:     22.20
```

As the package size increases, a higher bandwidth is observed. This is because, for smaller packages, the overhead of initiating P2P data transfers dominates the total transfer time.

With B60 Pro and PCIe Gen5 x8 connection, unidirectional write operations between two devices can typically reach up to ~20 GB/s, while bi-directional writes can achieve up to 40 GB/s.

If 2 GPUs are under different CPU sockets, the P2P throughput would be impacted by the UPI connection as well.

Besides the default test, you can specify the following options to customize your test.

Table 2. ze_peer Command Option

Option	Description
-t transfer_bw	Run bandwidth benchmark. The unit is gigabytes per second.

-t latency	Run latency benchmark. Internally, it will run multi-round tests for one package size, then calculate the average time required for one package as latency.
-b	Run a bi-directional test between 2 devices. The total data transferred will double the package size.
-u	Specify the engines. Accepts a comma-separated list of engines when running parallel tests with multiple targets. If there is only one target, you can only specify one engine.
-z	Specify the package size in bytes.
-i	Number of iterations to run. Default: 50.
-o	Operation to perform. "read" for reading from remote. "write" for writing to remote.
-v	Validate data of P2P transfer (only 1 iteration is executed). The time spent on validation would not be counted in the bandwidth/latency calculation.
-c	Run continuously until hitting CTRL+C
-d	Comma-separated list of destination devices.
-s	Comma-separated list of source devices.

Some pipeline examples are as follows:

1. Run the BW test for only 128 MB using the first engine with id 0 and write operation.
   ```
   ./ze_peer -t transfer_bw -z 134217728 -o write
   ```

2. Run the BW test for only 128 MB with write operation from device 1 to device 2.
   ```
   ./ze_peer -t transfer_bw -z 134217728 -o write -s 1 -d 2
   ```

3. Run the BW test for only 128 MB with write operation from device 1 to device 2 with engine id 1 of device 1.
   ```
   ./ze_peer -t transfer_bw -z 134217728 -o write -s 1 -d 2 -u 1
   ```

4. Run the BW test for only 128 MB with bi-directional write operation from device 1 to device 2. The total transferred data size would be 512 MB.
   ```
   ./ze_peer -t transfer_bw -z 134217728 -o write -s 1 -d 2 -b
   ```

5. Run the BW test for only 128 MB with write operation from devices 0 & 1 to device 2
   ./ze_peer -t transfer_bw -z 134217728 -o write -s 0,1 -d 2

4.3 Parallel Test

There are some modes to perform parallel P2P operation with the patterns below:

- 1 source to 1 target with all engines
- 1 source to N targets with all engines
- multiple 1 to 1 source/target pairs with all engines

The benefit of the parallel mode is that it enhances P2P performance by utilizing all available engines within the GPU to operate concurrently. If you want to get the best P2P throughput, you may try these modes.

Table 3. Parallel Test Options

Option	Description
--parallel_single_target	Divide the buffer into the number of engines passed with option -u, and perform parallel copies using those engines from the source passed with option -s to a single target specified with option -d. By default, a copy is made from device 0 to device 1 using all available engines with compute capability.
--parallel_multiple_targets	Perform parallel copies from the source passed with option s to the targets specified with option -d, each one using a separate engine specified with option -u. By default, a copy is made from device 0 to all other devices, using all available engines with the compute capability.
--parallel_pair_targets	Perform parallel copies from pairs of the source and targets each one using a separate engine specified with option -u. Accepts a comma-separated list of paired devices, with source and destination pairs defined with a colon. Example pair list: src1:dst1,src2:dst2
--divide_buffers	For parallel multiple targets test, divide buffers across available engines specified with option -u. Can be used for -parallel_multiple_targets and --parallel_pair_targets. Not required for --parallel_single_target since it will divide buffers by default.

Below are some examples.

1. Run parallel_single_target test, between devices 0 and 1, using engines 0 and 1, for a size of 128 MB. In this case, the buffer is divided into the number of engines (2), for a resulting chunk size of 64 MB. Then, two copies are submitted in parallel, per chunk, and per engine.

```
./ze_peer --parallel_single_target -t transfer_bw -z 134217728 -s 0
-d 1 -u 0,1 -o write
```

2. Run parallel_multiple_targets test, with source device 1, and destination devices 2 and 3, using engines 0 and 1, for a 128 MB buffer. In this case, two allocations of such size are created. Then, two copies are submitted in parallel, one between devices 1 and 2, and the other between devices 1 and 3, using engines 0 and 1, respectively.

```
./ze_peer --parallel_multiple_targets -t transfer_bw -z 134217728 -s
1 -d 2,3 -u 0,1
```

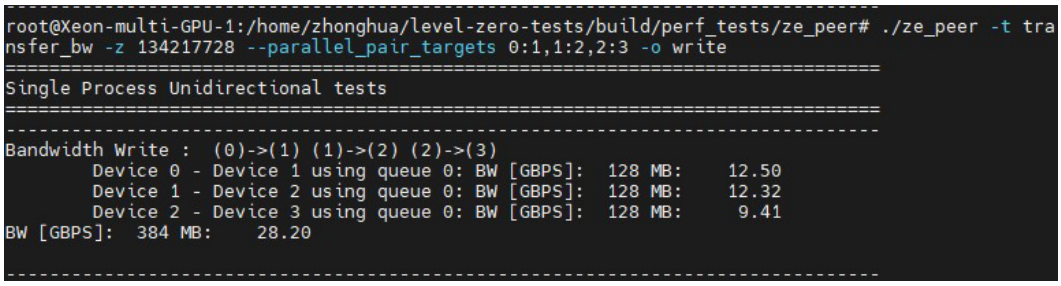
3. The same as #2, just that the 128 MB buffer would be divided into the number of engines for each copy by utilizing all engines specified by -u (0 and 1) to copy each chunk.

```
./ze_peer --parallel_multiple_targets -t transfer_bw -z 134217728 -s
1 -d 2,3 -u 0,1 --divide_buffers -o write
```

4. Run parallel_pair_targets test, with 3 pairs between device 0 and 1, device 1 and 2, device 2 and 3, for a 128 MB buffer per pair. In this case, 3 allocations of such size are created. Then, 3 copies are submitted in parallel using the first engine of the source GPU per pair.

```
./ze_peer -t transfer_bw -z 134217728 --parallel_pair_targets
0:1,1:2,2:3 -o write
```

Figure 7. parallel_pair_targets Test with 3 Pairs



5. The same as #4, just using different engines, 1st pair using engine 0, 2nd pair using engine 1.

```
./ze_peer -t transfer_bw -z 134217728 --parallel_pair_targets
0:1,1:2 -u 0,1 -o write
```

6. The same as #4, just each pair would use all engines specified by -u (0,1) on the source GPU to copy data with 1/2 of 128 MB for each engine.

```
./ze_peer -t transfer_bw -z 134217728 --parallel_pair_targets
0:1,1:2 -u 0,1 --divide_buffers -o write
```

4.4 Stability Test

Since this solution focuses on the LLM inference workload with multiple Intel® Arc™ cards, collective communication cross is often used for stability benchmark.

Assuming you have 4 cards, ze_peer can simulate ring rebased AllReduce operation with the command below. It will continue to run until you press "Ctrl+C".

```
./ze_peer -t transfer_bw  --parallel_pair_targets 0:1,1:2,2:3,3:0 -z
134217728 -o write -v -c
```

Figure 8. Run Stability Test with AllReduce-like Operation

```
root@Xeon-multi-GPU-1:/home/zhonghua/level-zero-tests/build/perf_tests/ze_peer# ./ze_peer -t tra
nsfer_bw  --parallel_pair_targets 0:1,1:2,2:3,3:0 -z 134217728 -o write
================================================================================
Single Process Unidirectional tests
================================================================================
--------------------------------------------------------------------------------
Bandwidth Write :  (0)->(1) (1)->(2) (2)->(3) (3)->(0)
        Device 0 - Device 1 using queue 0: BW [GBPS]:  128 MB:    20.06
        Device 1 - Device 2 using queue 0: BW [GBPS]:  128 MB:    20.06
        Device 2 - Device 3 using queue 0: BW [GBPS]:  128 MB:    19.89
        Device 3 - Device 0 using queue 0: BW [GBPS]:  128 MB:    19.87
BW [GBPS]:  512 MB:    79.37

--------------------------------------------------------------------------------
```

It will show the transfer bandwidth of each pair (~20 GB/s), and also show the bandwidth of a total 512 MB data as 79.37 GB/s.

When "-c" option added, there is a known issue that the bandwidth of each pair is not correct, but the total bandwidth is correct.

Figure 9. Known Issue with "-c" Option

```
        Device 0 - Device 1 using queue 0: BW [GBPS]:  128 MB:     0.71
        Device 1 - Device 2 using queue 0: BW [GBPS]:  128 MB:     0.71
        Device 2 - Device 3 using queue 0: BW [GBPS]:  128 MB:     0.70
        Device 3 - Device 0 using queue 0: BW [GBPS]:  128 MB:     0.70
BW [GBPS]:  512 MB:    80.01
```

§

5.0 Memory Bandwidth Tests

Two sub-tests of ze_peak can be used for the memory bandwidth benchmark. “ransfer_bw” for H2D (Host-to-Device)/D2H (Device-to-Host). “global bw” for D2D (Device-to-Device).

Figure 10. ze_peak Help Menu

```
edgeai@edgeaihost10:~/james/multi-arc-bmg-offline-installer-25.32.1.2/tools/level-zero-tests$ ./ze_peak -h

ze_peak [OPTIONS]

OPTIONS:
  -r, --driver num            choose driver (num starts with 0)
  -d, --device num            choose device   (num starts with 0)
  -e                          time using ze events instead of std chrono timer
                              hide driver latencies [default: No]
  -t, string                  selectively run particular tests
      global_bw               selectively run global bandwidth test
      hp_compute              selectively run half precision compute test
      sp_compute              selectively run single precision compute test
      dp_compute              selectively run double precision compute test
      int_compute             selectively run integer compute test
      transfer_bw             selectively run transfer bandwidth test
      kernel_lat              selectively run kernel latency test
```

5.1 H2D/D2H Tests

Below is the test result of “transfer_bw” on Intel B60 Pro GPU, which is connected to the host with PCIe Gen5 x8. The unidirectional speed of the PCIe bus is 32GB/s. By considering the 128/130 bit encoding cost and B60 Pro GPU supported PCIe TLP size by EU (Execution Unit), the payload bandwidth in theory on the PCIe bus is 24GB/s. On B60 Pro GPU, 22.7GB/s D2H speed is reasonable.

Figure 11. ze_peak transfer_bw result

```
root@edgeaihost10:/home/edgeai/james/multi-arc-bmg-offline-installer-25.32.1.2/tools/level-zero-tests# ./ze_peak -t transfer_bw
Device :
 * name : Intel(R) Graphics [0xe211]
 * vendorId : 8086
 * deviceId : e211
 * subdeviceId : 0
 * isSubdevice : FALSE
 * UUID : 00000000-0000-0019-0000-0000e2118086
 * coreClockRate : 2400
 * maxMemAllocSize : 24385683456 bytes

Transfer Bandwidth (GB/s)
enqueueWriteBuffer : 27.5755 GB/s
enqueueReadBuffer : 22.7376 GB/s
GPU Copy Host to Shared Memory : 27.5754 GB/s
GPU Copy Shared Memory to Host : 22.737 GB/s
System Memory Copy to Shared Memory : 9.75944 GB/s
System Memory Copy from Shared Memory : 9.75818 GB/s
<<<<<<<<<<<<<<<<<<<<<<<<<<<<
```

Assume the TLP header size is 20B, below is how to calculate the bandwidth with 64B payload size.

Figure 12. PCIe Bandwidth Calculation

PCIe BW in theory

Gen	Speed per Lane	Lane #	Encoding	BW in theory (uni-directional)	BW in theory (bi-directional)
4	16 GT/s	x16	128b/130b	31.5 GB/s	63 GB/s
5	32 GT/s	x8	128b/130b	31.5 GB/s	63 GB/s

Gen5 x8，Payload 64B，Header 20B

$$E = \frac{64}{64+20} = \frac{64}{84} \approx 0.7619$$

$$B_{eff} = 31.5 \times 0.7619 \approx \boxed{24.0\ \text{GB/s}}$$

“enqueue buffer” equals “GPU Copy” operation, below is a table to show the details of GPU copy tests.

Table 4. ze_peak GPU Copy Details

GPU Copy	Shared Memory	Operation Submitter	API Used	Operation on PCIe
Host to Shared Memory	GDDR	GPU	zeCommandListAppendMemoryCopy	PCIe Read
Shared Memory to Host	GDDR	GPU	zeCommandListAppendMemoryCopy	PCIe Write

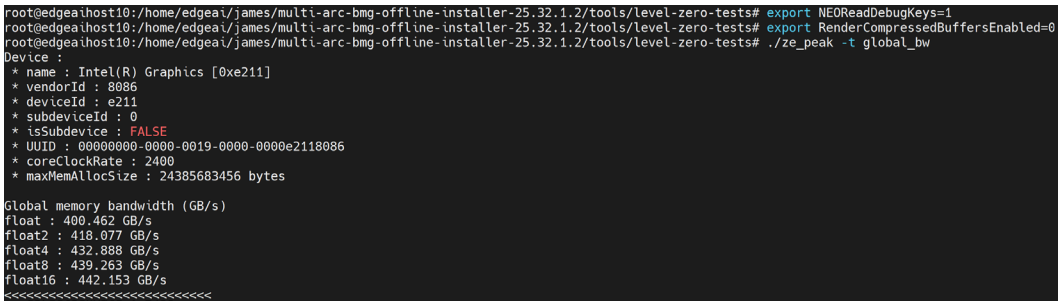
Since the Maximum Read Request Size (MRRS) is typically larger than the Maximum Payload Size (MPS), and writes from the GPU to shared memory are generally faster than writes to system memory due to cache-coherency overhead in the fabric, the overall host-to-device (H2D) transfer speed is higher than the device-to-host (D2H) speed.

“System Memory Copy” operations are executed by CPU, thus with lower speed.

5.2 D2D Tests

Below is the test result of “global_bw” on B60 Pro GPU with PCIe gen5 x8 connection.

Figure 13. ze_peak global_bw result



Make sure to export 2 environment variables: NEOReadDebugKeys=1 and RenderCompressdBuffersEnabled=0. If not, the bandwidth would be doubled. The memory bandwidth of B60 in theory is 456GB/s, the benchmark data above is 442GB/s which is reasonable.

§

6.0 oneCCL Tests

This chapter includes the test of oneCCL based collective communication performance with different package sizes and algorithms.

As shared above in the Multi-GPU Get Started Guide, 1ccl_benchmark binary and dependency already installed if you follow this guide.

6.1 Options

Below are the 1ccl_benchmark options descriptions.

Table 5. oneCCL benchmark Command Option

Option	Description
-a	Specify the type of the SYCL device for running collective communication operations. The possible values are ``cpu``, and ``gpu``.
-m	Specify the type of SYCL memory type used for buffer. The possible values are ``usm`` (unified shared memory) and ``buf`` (buffers on CPU or GPU).
-u	When the memory type is USM, further specify the type of USM. The possible values are ``device`` or ``shared``. ``device`` means that memory only exists on the device (such as GPU HBM), ``shared`` means memory can be directly accessed by both hosts and devices.
-e	Specify the type of the SYCL queue. The possible values are ``in_order``(Ordered Queue) and ``out_order``(Disordered queue).
-l	Specify the collective communication operator to run. Accept a comma-separated list, without whitespace characters, of collectives to run. The available collectives are ``allreduce``, ``allgather``, ``alltoall``
-i	Specify the number of iterations executed by the benchmark.
-w	Specify the number of warmup iterations. It means the number of iterations the benchmark runs before starting the timing of the iterations specified with the ``-i`` argument.
-f	Specify the minimum number of elements used for the collective.
-t	Specify the maximum number of elements for the collective. The benchmark will be executed from the minimum to the maximum element size, increasing by a factor of 2 at each step.
-j	Specify the iteration policy. Possible values are off and auto. When the iteration policy is off, the number of iterations is the same across the message sizes. When the iteration policy is auto, the number of iterations reduces based on the message size of the collective operation.
-p	Specify whether to use persistent collectives (``p=1``) or not (``p=0``).

	.. note:: A collective is persistent when the same collective is called with the same parameters multiple times. oneCCL generates a schedule for each collective it runs and can apply optimizations when persistent collectives are used. It means the schedule is generated once and reused across the subsequent invocations, saving the time to generate the schedule.
-d	Specify the datatype. Accept a comma-separated list, without whitespace characters, of datatypes to benchmark. The available types are ``float16``, ``float32``, and ``bfloat16``.
-q	Specify for oneCCL to use in-place (``1``) or out-of-place (``0``) buffers. With the in-place buffers, the send and receive buffers used by the collective are the same. With the out-of-place, the buffers are different.
-o	file to store CSV-formatted data into

6.2 Command Examples

Run 4 cards allreduce test, element size between 512 and 67108864, using float16, then save the results to result.csv.

```
mpirun -np 4 /usr/bin/1ccl_benchmark -a gpu -m usm -u device -e
in_order -l allreduce -i 50 -w 20 -f 512 -t 67108864 -j off -p 0 -d
float16 -q 0 -o result.csv
```

The result output is as shown below:

Figure 14. allreduce results with 4 B60 Pro

```
datatype: float16
reduction: sum

#------------------------------------------------------------
# Benchmarking: allreduce
# #processes: 4
#------------------------------------------------------------

       #bytes   #elem_count  #repetitions   t_min[usec]   t_max[usec]   t_avg[usec]  stddev[%]   algbw[GB/s]   busbw[GB/s]
         1024           512            50         52.00         52.27         52.11       0.20          0.02          0.03
         2048          1024            50         54.29         54.45         54.37       0.14          0.04          0.06
         4096          2048            50         51.01         51.21         51.08       0.16          0.08          0.12
         8192          4096            50         52.27         52.34         52.31       0.05          0.16          0.23
        16384          8192            50         50.84         51.00         50.91       0.12          0.32          0.48
        32768         16384            50         43.70         43.99         43.83       0.26          0.75          1.12
        65536         32768            50         41.56         41.68         41.61       0.11          1.58          2.36
       131072         65536            50         42.87         42.96         42.91       0.09          3.05          4.58
       262144        131072            50         47.84         48.02         47.94       0.14          5.47          8.20
       524288        262144            50         64.00         64.11         64.05       0.07          8.19         12.28
      1048576        524288            50         98.49         98.65         98.56       0.06         10.64         15.96
      2097152       1048576            50        171.23        171.26        171.24       0.01         12.25         18.37
      4194304       2097152            50        322.87        322.97        322.91       0.01         12.99         19.48
      8388608       4194304            50        682.67        682.73        682.70       0.00         12.29         18.43
     16777216       8388608            50       1270.65       1270.70       1270.67       0.00         13.20         19.81
     33554432      16777216            50       2537.39       2537.44       2537.42       0.00         13.22         19.84
     67108864      33554432            50       5061.13       5061.15       5061.14       0.00         13.26         19.89
    134217728      67108864            50      10079.76      10079.79      10079.77       0.00         13.32         19.97

# All done
```

With B60 Pro GPU and PCIe Gen5 x8 connection, allreduce transfer bandwidth reaches up to ~20GB/s.

Run 4 cards allgather test, element size between 512 and 67108864, using float16, then save the results to result.csv

```
mpirun -np 4 /usr/bin/1ccl_benchmark -a gpu -m usm -u device -e
in_order -l alltoall -i 50 -w 20 -f 512 -t 67108864 -j off -p 0 -d
float16 -q 0 -o result.csv
```

The result output is as shown below:

Figure 15. allgather results with 4 B60 Pro

```
datatype: float16
reduction: sum

#------------------------------------------------------------
# Benchmarking: allgather
# #processes: 4
#------------------------------------------------------------

       #bytes   #elem_count  #repetitions   t_min[usec]   t_max[usec]   t_avg[usec]  stddev[%]   algbw[GB/s]   busbw[GB/s]
         1024           512            50         36.15         36.31         36.20       0.17          0.11          0.08
         2048          1024            50         36.71         36.88         36.76       0.18          0.22          0.17
         4096          2048            50         41.27         41.48         41.36       0.20          0.40          0.30
         8192          4096            50         45.95         46.10         46.01       0.12          0.71          0.53
        16384          8192            50         41.83         42.05         41.96       0.19          1.56          1.17
        32768         16384            50         37.69         37.85         37.74       0.16          3.47          2.60
        65536         32768            50         34.31         34.45         34.36       0.16          7.63          5.72
       131072         65536            50         41.17         41.34         41.24       0.15         12.71          9.54
       262144        131072            50         59.58         59.79         59.69       0.13         17.57         13.18
       524288        262144            50         93.10         93.22         93.16       0.05         22.51         16.88
      1048576        524288            50        164.41        164.44        164.43       0.01         25.51         19.13
      2097152       1048576            50        314.24        314.35        314.29       0.01         26.69         20.02
      4194304       2097152            50        630.16        630.28        630.23       0.01         26.62         19.97
      8388608       4194304            50       1222.51       1222.54       1222.53       0.00         27.45         20.59
     16777216       8388608            50       2433.35       2433.40       2433.37       0.00         27.58         20.68
     33554432      16777216            50       4851.63       4851.65       4851.64       0.00         27.66         20.75
     67108864      33554432            50       9680.21       9680.31       9680.25       0.00         27.73         20.80
    134217728      67108864            50      19351.95      19352.04      19351.99       0.00         27.74         20.81

# All done
```

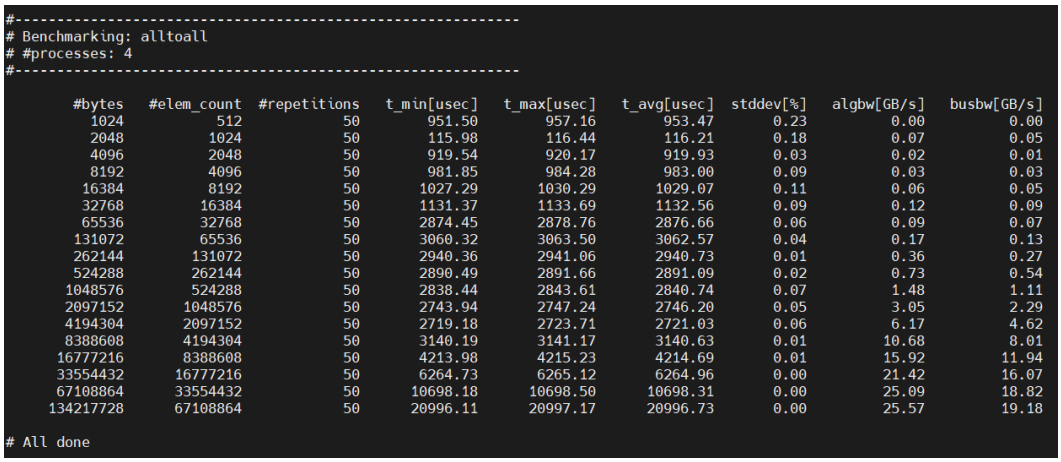
With B60 Pro GPU and PCIe Gen5 x8 connection, allgather transfer bandwidth reaches up to ~20GB/s.

Run 4 cards alltoall test, element size between 512 and 67108864, using float16, then save the results to result.csv

```
mpirun -np 4 /usr/bin/1ccl_benchmark -a gpu -m usm -u device -e
in_order -l alltoall -i 50 -w 20 -f 512 -t 67108864 -j off -p 0 -d
float16 -q 0 -o result.csv
```

The result output is as shown below:

Figure 16. alltoall results with 4 B60 Pro



```
#---------------------------------------------------------
# Benchmarking: alltoall
# #processes: 4
#---------------------------------------------------------

       #bytes   #elem_count  #repetitions    t_min[usec]    t_max[usec]    t_avg[usec]  stddev[%]   algbw[GB/s]   busbw[GB/s]
         1024           512            50         951.50         957.16         953.47       0.23          0.00          0.00
         2048          1024            50         115.98         116.44         116.21       0.18          0.07          0.05
         4096          2048            50         919.54         920.17         919.93       0.03          0.02          0.01
         8192          4096            50         981.85         984.28         983.00       0.09          0.03          0.03
        16384          8192            50        1027.29        1030.29        1029.07       0.11          0.06          0.05
        32768         16384            50        1131.37        1133.69        1132.56       0.09          0.12          0.09
        65536         32768            50        2874.45        2878.76        2876.66       0.06          0.09          0.07
       131072         65536            50        3060.32        3063.50        3062.57       0.04          0.17          0.13
       262144        131072            50        2940.36        2941.06        2940.73       0.01          0.36          0.27
       524288        262144            50        2890.49        2891.66        2891.09       0.02          0.73          0.54
      1048576        524288            50        2838.44        2843.61        2840.74       0.07          1.48          1.11
      2097152       1048576            50        2743.94        2747.24        2746.20       0.05          3.05          2.29
      4194304       2097152            50        2719.18        2723.71        2721.03       0.06          6.17          4.62
      8388608       4194304            50        3140.19        3141.17        3140.63       0.01         10.68          8.01
     16777216       8388608            50        4213.98        4215.23        4214.69       0.01         15.92         11.94
     33554432      16777216            50        6264.73        6265.12        6264.96       0.00         21.42         16.07
     67108864      33554432            50       10698.18       10698.50       10698.31       0.00         25.09         18.82
    134217728      67108864            50       20996.11       20997.17       20996.73       0.00         25.57         19.18

# All done
```

With B60 Pro GPU and PCIe Gen5 x8 connection, alltoall transfer bandwidth reaches up to ~19GB/s.

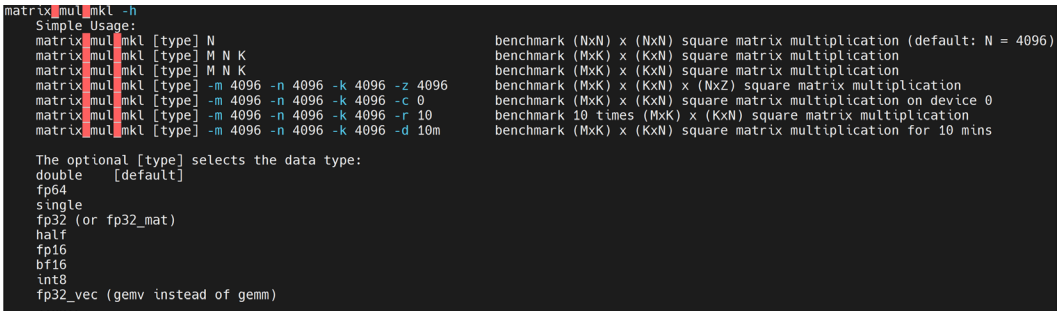
§

7.0 GEMM Tests

Matrix Multiplication uses oneMKL to multiply two large matrices and measure device performance. The benchmark binary named matrix_mul_mkl is already installed in the /usr/bin directory. You can directly use it if you follow Multi-GPU get started guide to setup environment.

7.1 matrix_mul_mkl usage

Figure 17. matrix_mul_mkl usage



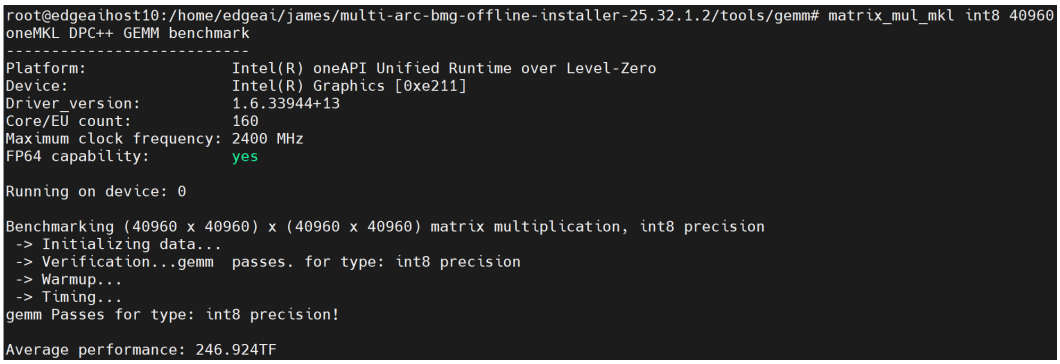
You can use "-h" to check the help information. It supports double(fp64), single(fp32), half(fp16), bf16, int8 datatype. It will run in GPU automatically if sycl-ls can find the GPU. Otherwise, it will run on the CPU.
You need to source the oneAPI environment variable first, as below:

```
$ source /opt/intel/oneapi/setvars.sh
```

7.2 Run matrix_mul_mkl

In the below example, matrix_mul_mkl run int8 matrix multiplication on B60 Pro GPU, got 246TOPS. By default, it will run benchmark on all available GPUs.

Figure 18. matrix_mul_mkl on B60 with int8



In matrix_mul_mkl, the benchmark runs multiple times. The result of each run is compared against the result of the first run. If no differences are found, it reports “gemm PASSES xxx”; otherwise, it reports “gemm FAILS xxx”. When all runs are correct, the average performance is displayed.

§